UNIVERSIDAD TECNOLÓGICA NACIONAL
Tecnicatura Universitaria en Programación

TRABAJO PRÁCTICO - UNIDAD 6

Mantenimiento, respaldo y recuperación

Proyecto integrador: Food Store

Plan de mantenimiento · Respaldo · Simulacro de recuperación

Materia: Base de Datos II
Carrera: Tecnicatura Universitaria en Programación (UTN)

Autores:
Lemus Nahuel
Anglat Juan
Elias Juan Cruz

Año 2026

1. Parte 1 - Especificación y plan de mantenimiento

El trabajo continúa Food Store de las unidades anteriores. Se utilizó PostgreSQL 18.6 en Railway, proyecto postgrefacu, servicio Postgres. La base principal del laboratorio fue foodstore, copia de trabajo de railway. La recuperación se realizó en una base nueva llamada foodstore_restore_test; no se eliminó ni sobrescribió la fuente.

Objetivo y alcance de SPEC_U6.md

Construir un procedimiento automatizable que produzca un respaldo custom, capture el estado de la base, revise mantenimiento y pruebe una recuperación completa con validaciones. Las frecuencias siguientes son el plan propuesto: no hay una programación automática activa.

Tarea	Frecuencia propuesta	Condición de éxito y evidencia
Respaldo de foodstore	Diaria, 02:00 de Argentina	Archivo .dump fechado, pg_dump termina con 0, listado legible y log con conteos.
Chequeo de salud	Diaria, después del respaldo	Consultas ejecutadas; revisión de tamaño, conexiones, tablas y actividad.
Revisión de mantenimiento	Semanal	Revisar autovacuum, tuplas muertas y estadísticas; actuar solo con justificación.
Restauración de prueba	Semanal	Base nueva validada, conteos/contenido coincidentes y cero inválidos.

Objetivos de recuperación

RPO previsto: hasta 24 horas con frecuencia diaria. RTO objetivo: 120 segundos para este conjunto pequeño de datos. El RTO del ensayo se mide desde inmediatamente antes de CREATE DATABASE hasta terminar las validaciones. La antigüedad del respaldo al inicio aproxima la ventana de exposición para discutir el RPO; no es el tiempo que tarda pg_restore.

Reglas de seguridad y revisión

Regla	Aplicación
Conservar la fuente	Nunca restaurar sobre foodstore ni borrar sus datos.
Destino único y descartable	Si foodstore_restore_test existe, abortar. No usar DROP, --clean ni terminar sesiones.
Sin contraseñas en la entrega	Credenciales efímeras desde Railway CLI o entorno; no se incluyen URL privadas ni secretos.
Errores verificables	Fallo de pg_dump, pg_restore o validación implica código distinto de cero y diagnóstico.
Mantenimiento seguro	Sin VACUUM FULL periódico; ANALYZE solo cuando las estadísticas lo justifican.

Archivos y asistencia

SPEC_U6.md y AGENTS.md fijan las reglas; backup_foodstore.ps1 y restore_test.ps1 invocan laboratorio_u6.py, que utiliza el cliente compartido laboratorio/railway_client.py. health_check.sql y maintenance.sql contienen las consultas. La asistencia efectiva fue de Codex. OpenCode se intentó ejecutar pero falló por un esquema incompatible de herramientas (HTTP 400); su intervención válida y la revisión humana exigidas siguen pendientes.

2. Parte 2 - Respaldo automatizable

Script probado: backup_foodstore.ps1

```
$ErrorActionPreference = 'Stop'
python "$PSScriptRoot/laboratorio_u6.py" backup
if ($LASTEXITCODE -ne 0) { throw "El respaldo fallo (codigo $LASTEXITCODE). Ver logs." }
```

El wrapper detiene el flujo si el programa Python falla. El orquestador genera un nombre con fecha y hora UTC, ejecuta pg_dump en formato custom y comprueba que pg_restore pueda listar el archivo. El diagnóstico se registra en logs/; el .dump se conserva localmente, excluido del repositorio y del paquete de entrega.

Comandos y consistencia del corte

```
pg_dump -d foodstore -Fc --snapshot SNAPSHOT_EXPORTADO -f backups/foodstore_FECHA.dump
pg_restore --list backups/foodstore_FECHA.dump
```

Host, puerto, usuario y contraseña se suministran por el entorno del proceso cliente. El programa abre una transacción REPEATABLE READ de solo lectura, exporta su snapshot y lo mantiene activo mientras pg_dump lo utiliza. Los conteos y resúmenes de contenido se obtienen del mismo snapshot, evitando comparar dos estados distintos de la fuente.

Evidencia real de generación

Dato	Resultado registrado
Archivo	foodstore_20261020T205205_768821Z.dump
Formato	Custom (-Fc)
Inicio local (UTC-03:00)	20/10/2026 17:52:05
Fin local (UTC-03:00)	20/10/2026 17:52:36
Tamaño	321.577 bytes
Salida y lectura del dump	pg_dump: 0; pg_restore --list: correcto
Snapshot compartido	Sí, entre la evidencia de referencia y pg_dump

Registro y conservación

```
logs/backup_20261020T205205_768821Z.json
logs/20261020T205205_768821Z_contenido_dump.txt
backups/foodstore_20261020T205205_768821Z.dump
```

El log contiene nombre de archivo, marcas temporales, tamaño, código de salida, conteos y resúmenes MD5. No guarda filas personales ni credenciales. MD5 se emplea para comparar contenido en este ensayo, no como garantía criptográfica contra alteraciones maliciosas.

El respaldo utilizado precede a la reconstrucción de identidades sintéticas de U5. Las mediciones de esta entrega corresponden a ese archivo concreto y no afirman recuperar cambios posteriores ni roles globales del servidor.

3. Parte 3 - Monitoreo y mantenimiento

Cuatro controles de health_check.sql

```
SELECT clock_timestamp() AS instante, current_database() AS base,
       pg_size_pretty(pg_database_size(current_database())) AS tamano;
SELECT datname, numbackends, xact_commit, xact_rollback, deadlocks, stats_reset
FROM pg_stat_database WHERE datname = current_database();
SELECT relname, n_live_tup, n_dead_tup, n_mod_since_analyze,
       last_vacuum, last_autovacuum, last_analyze, last_autoanalyze
FROM pg_stat_user_tables ORDER BY n_dead_tup DESC, relname;
SELECT pid, usename, application_name, state, clock_timestamp()-query_start AS duracion,
       wait_event_type, wait_event, left(query,160) AS consulta
FROM pg_stat_activity
WHERE datname=current_database() AND state <> 'idle' AND pid<>pg_backend_pid();
```

Control	Observación del laboratorio
Tamaño	14 MB en el instante del chequeo.
Conexiones y transacciones	Una conexión propia; 170 commits, 5 rollbacks y 0 deadlocks en la captura.
Tablas y estadísticas	Estadísticas presentes; n_mod_since_analyze = 0 en las relaciones observadas.
Consultas activas	Sin otras consultas activas al muestrear. No equivale a monitoreo continuo.

Revisión y decisión de mantenimiento

Autovacuum estaba activo. Los umbrales observados fueron 50 + 0,1 × n_live_tup para ANALYZE y 50 + 0,2 × n_live_tup para VACUUM. Se encontraron muy pocas tuplas muertas y no había modificaciones pendientes de análisis. La regla condicional no seleccionó tablas: no se justificaba repetir ANALYZE en esa ejecución.

```
SELECT format('ANALYZE %I.%I;',schemaname,relname)
FROM pg_stat_user_tables
WHERE (last_analyze IS NULL AND last_autoanalyze IS NULL)
 OR n_mod_since_analyze >= current_setting('autovacuum_analyze_threshold')::numeric
    + current_setting('autovacuum_analyze_scale_factor')::numeric*n_live_tup
\gexec
```

La acción segura es ANALYZE condicionado: si faltan estadísticas después de una carga/restauración o se supera el umbral, se actualizan las estadísticas del planificador. Si no se cumple la condición, no se ejecuta. No se utilizó VACUUM FULL, porque reescribe tablas y requiere un bloqueo fuerte; no corresponde al mantenimiento periódico de este laboratorio.

Evidencias: logs/health.txt y logs/maintenance.txt contienen las salidas del servidor antes y después de la evaluación. Los contadores de pg_stat_database son acumulados desde su reinicio; no se interpretan como transacciones exclusivas de esta prueba.

4. Parte 4 - Simulacro de recuperación contra reloj

Procedimiento y control del destino

Se seleccionó el respaldo identificado en la Parte 2. El programa consultó pg_database para comprobar que foodstore_restore_test no existía. La copia de un ensayo anterior ya estaba conservada con otro nombre. El cronómetro se inició inmediatamente antes de crear la base nueva.

```
CREATE DATABASE foodstore_restore_test TEMPLATE template0;
pg_restore -d foodstore_restore_test --exit-on-error --no-owner --no-privileges \
  backups/foodstore_20261020T205205_768821Z.dump
```

La restauración termina ante el primer error. Después se obtienen conteos y resúmenes del contenido, se comprueban productos/detalles inválidos y se cuentan las restricciones restauradas. Si alguna comprobación falla, el programa conserva el destino para diagnóstico y termina con error. No elimina automáticamente una base anterior.

Controles mínimos de validación

```
SELECT 'categoria' tabla, count(*) filas FROM categoria
UNION ALL SELECT 'cliente',count(*) FROM cliente
UNION ALL SELECT 'producto',count(*) FROM producto
UNION ALL SELECT 'pedido',count(*) FROM pedido
UNION ALL SELECT 'detalle_pedido',count(*) FROM detalle_pedido;

SELECT count(*) FROM producto WHERE precio<0 OR stock<0;
SELECT count(*) FROM detalle_pedido WHERE cantidad<=0 OR precio_unitario<0;
```

Tabla	Antes del respaldo	Después de restaurar	Contenido
categoria	20	20	Coincide
producto	1000	1000	Coincide
cliente	5000	5000	Coincide
pedido	20000	20000	Coincide
detalle_pedido	20000	20000	Coincide

Comprobación adicional	Resultado
Productos inválidos	0
Detalles inválidos	0
Restricciones restauradas	18
Comparación de contenido	Resúmenes MD5 coincidentes en las cinco tablas
Resultado global	Correcto (ok = true)

Mediciones observadas

Inicio: 20/10/2026 17:56:29 (UTC-03:00). Fin: 20/10/2026 17:57:01 (UTC-03:00). RTO: 32,224 segundos. Antigüedad del respaldo: 263,558 segundos, equivalentes a 4 minutos y 23,558 segundos. El RTO incluye comunicación con Railway y todas las validaciones del orquestador.

La validación cumple los mínimos del TP y agrega una comparación del contenido. No reemplaza pruebas funcionales completas de toda una aplicación ni un ensayo de recuperación del clúster. Evidencia completa: logs/restore_resultado.json.

5. Parte 5 - Informe post-incidente

Fecha del ensayo: 20/10/2026. Entorno: PostgreSQL 18.6 en Railway, proyecto postgrefacu. El simulacro recuperó una copia de Food Store en una base nueva; no existió un incidente real ni se sobrescribió foodstore.

Respaldo utilizado

Archivo: foodstore_20261020T205205_768821Z.dump. Inicio: 20/10/2026 17:52:05 (Argentina, UTC-03:00). Formato custom, 321.577 bytes, snapshot compartido con la referencia. pg_dump terminó con código 0 y el listado de pg_restore resultó legible.

RPO y RTO: previstos y observados

Indicador	Previsto	Observado / interpretación
RTO	120 segundos	32,224 segundos desde antes de CREATE DATABASE hasta finalizar las validaciones. Objetivo cumplido.
RPO	Hasta 24 horas con backup diario	Respaldo de 263,558 segundos de antigüedad al iniciar. Aproxima la ventana de exposición; no mide transacciones perdidas.

La recuperación finalizó correctamente a las 20/10/2026 17:57:01 (UTC-03:00). Las frecuencias son propuestas y todavía no están programadas: no se afirma una garantía operativa de RPO de 24 horas.

Validaciones y resultado

Se recuperaron 20 categorías, 1.000 productos, 5.000 clientes, 20.000 pedidos y 20.000 detalles. Los conteos y resúmenes de contenido coincidieron en las cinco tablas. Productos inválidos: 0; detalles inválidos: 0; restricciones restauradas: 18. El ensayo no incluye las identidades de U5 agregadas con posterioridad al respaldo ni los roles globales del servidor.

Problema observado y mejora propuesta

Existía un ensayo previo, conservado con otro nombre. La condición de abortar si foodstore_restore_test existe evita reemplazarlo accidentalmente. Como mejora se propone una rotación supervisada de ensayos, respaldo cifrado fuera del servidor y alertas ante fallos. Esto reduce riesgo de sobrescritura y permite detectar que un respaldo diario no se completó.

Asistencia, verificación y pendiente humano

La revisión y ejecución de scripts se realizaron con Codex; se verificaron códigos de salida, listado del dump, salud y validaciones de restauración. Se intentó OpenCode, pero falló con HTTP 400 por un esquema incompatible de herramientas. No se atribuye a OpenCode el trabajo efectuado: quedan pendientes su intervención válida y la revisión humana del estudiante exigidas por la consigna.

Reproducción y evidencias de la entrega

```
powershell -File food-store-tp-u6/backup_foodstore.ps1
python food-store-tp-u6/laboratorio_u6.py health
python food-store-tp-u6/laboratorio_u6.py maintenance
powershell -File food-store-tp-u6/restore_test.ps1
```

Si el destino ya existe, el último paso debe detenerse; no borrarlo para repetir sin una decisión expresa. La carpeta de entrega contiene SPEC_U6.md, AGENTS.md, scripts, el cliente compartido y logs. El archivo .dump se omite del paquete; su creación, tamaño y recuperación efectiva están acreditados.

Referencias: PostgreSQL 18 - Backup and Restore; pg_dump; pg_restore; Monitoring Database Activity; Routine Database Maintenance Tasks. Documentación oficial: https://www.postgresql.org/docs/18/backup.html y secciones relacionadas.